

Agent Interoperability through Safe and Bounded Capability Exposure: A MiSArch Case Study with GraphQL, MCP, and A2A

Zeyu Wang, Leitian Wang, Xuanyan Wang, Jou-En Lin, Haoyue Chen
Technische Universität Berlin
Berlin, Germany

Abstract—Modern autonomous agents require standardized mechanisms for discovering and invoking backend capabilities while maintaining security and interoperability. This paper investigates four progressively more sophisticated agent-facing architectures built on the MiSArch reference e-commerce platform, including direct GraphQL, Model Context Protocol (MCP), MCP with structured user profiles, and an Agent-to-Agent (A2A) architecture. We evaluate the four architectures under identical deployment conditions using representative recommendation and purchasing tasks. The evaluation considers end-to-end latency, communication complexity, interoperability, data sovereignty, and robustness against adversarial interactions. The results show that direct GraphQL provides the lowest latency, whereas MCP improves capability discoverability through standardized tool descriptions. The A2A architecture introduces explicit trust separation and stronger protection of user profiles while requiring only limited additional communication overhead. Security experiments further demonstrate that local policy enforcement remains necessary because protocol metadata alone cannot provide trustworthy security guarantees. Beyond the original evaluation, we report a fully closed-loop purchase workflow – the A2A purchase skill now drives MiSArch’s order and simulation services to a terminal `Order.orderStatus=PLACED / Payment.status=SUCCEEDED` outcome instead of stopping at a pending order – validated both locally and against a public Cloudflare deployment of the store-agent, together with a 15-scenario negative-path matrix and two additional deterministic security regressions (preview tampering, payment CVC leakage). These findings highlight the trade-offs between efficiency, interoperability, and trust, providing practical guidance for designing cloud-native agent systems.

Index Terms—agent interoperability, Model Context Protocol, Agent-to-Agent protocol, GraphQL, capability exposure, trust boundaries, secure tool use

I. INTRODUCTION

Recent advances in LLM-based agents have shifted software systems from traditional API consumers toward autonomous software entities capable of planning, reasoning, and tool use. As these systems become increasingly distributed, standardized interoperability protocols have become an essential component of cloud-native architectures.

Modern AI agents increasingly rely on external tools to perform real-world tasks, making structured capability discovery, backend services, and secure execution essential challenges for

cloud-native systems. While GraphQL provides flexible API access for developers, autonomous agents additionally require standardized tool descriptions, explicit execution semantics, and clear trust boundaries to interact safely with external services [1], [2].

As agent ecosystems become increasingly heterogeneous, interoperability is no longer limited to API compatibility. Autonomous agents must discover available capabilities, understand execution constraints, reason about potential side effects, and cooperate across organizational boundaries. These requirements motivate standardized interoperability protocols that support not only communication but also capability discovery, trust management, and safe interaction between autonomous systems [2], [3].

To investigate these challenges, we design a controlled four-arm experimental framework that progressively evolves from direct GraphQL access to MCP-based tool invocation, structured user profiles, and finally an Agent-to-Agent (A2A) architecture with explicit trust separation. This incremental design isolates the impact of protocol abstraction, profile representation, and multi-agent collaboration while preserving a common MiSArch backend.

Existing studies primarily introduce interoperability protocols or demonstrate their functionality in isolated scenarios [2], [3]. However, relatively little work systematically compares multiple agent-facing architectures under identical experimental conditions while jointly considering performance, interoperability, privacy, and security. This gap motivates the controlled comparative methodology adopted in this work.

Unlike prior studies that primarily compare interoperability protocols from a functional perspective [2], [3], this work systematically evaluates the architectural trade-offs introduced by different agent-facing interfaces. Beyond conventional performance metrics such as latency and task completion, we further investigate interoperability, capability discoverability, communication complexity, data sovereignty, and security against adversarial interactions.

The main contributions of this paper are summarized as follows:

- A controlled four-arm experimental framework for evaluating agent interoperability.
- An agent-facing extension of MiSArch supporting GraphQL, MCP, and A2A interaction, with the A2A store-agent implemented on the official `a2aproject/a2a-go/v2` SDK and its purchase skill closing the loop to a completed, locally simulated order and payment rather than a pending order.
- A comprehensive evaluation covering latency, interoperability, communication complexity, data sovereignty, and security, extended in this revision with a 15-scenario negative-path robustness matrix and two new deterministic attack regressions.
- A reproducible public deployment of the A2A store-agent (Cloudflare Worker + tunnel) demonstrating that the architecture’s trust-boundary claims hold across a real network boundary, not only on localhost.

Rather than proposing another interoperability protocol, this work aims to understand the architectural trade-offs introduced by different agent-facing interfaces and provide practical guidance for exposing cloud-native services to autonomous AI agents safely and efficiently.

The remainder of this paper is organized as follows. Section II reviews related work. Sections III–V present the system design, evaluation methodology, and experimental results, followed by discussion, limitations, and conclusions.

II. BACKGROUND AND RELATED WORK

This section reviews the literature most relevant to our work, including agent interoperability, GraphQL, MCP, A2A communication, and secure tool use.

A. Agent Interoperability

Large language model (LLM) agents combine reasoning, planning, and external tool use to accomplish complex tasks beyond traditional conversational systems [1]. Unlike conventional APIs designed for human developers, autonomous agents require discoverable capabilities, structured invocation, and explicit execution semantics to interact safely with backend services. Recent studies therefore emphasize standardized interoperability mechanisms that expose capabilities in a machine-readable and auditable manner [2], [4].

Compared with conventional service-oriented software, agent-oriented systems must support dynamic capability discovery and autonomous decision making during execution rather than relying on statically programmed workflows. Interoperability protocols increasingly describe not only communication formats but also executable capabilities and operational semantics, allowing agents to understand available functionality without prior programming [2].

B. MCP and A2A

The Model Context Protocol (MCP) standardizes agent-to-tool interaction through structured schemas and runtime metadata, enabling autonomous agents to discover and invoke tools consistently [2]. Complementarily, the Agent-to-Agent

(A2A) protocol enables communication and task delegation between autonomous agents operating across organizational or trust boundaries [3]. Together, these protocols provide complementary mechanisms for capability exposure and distributed agent collaboration.

Although both protocols improve interoperability, they address different architectural layers. MCP standardizes agent-to-tool interaction within a service domain, whereas A2A enables collaboration between autonomous agents operating across organizational and trust boundaries. Rather than competing approaches, the two protocols complement one another by supporting different forms of interaction in distributed agent ecosystems [3].

C. Secure Agent Tool Use

Recent studies have shown that exposing external tools to autonomous agents introduces new attack surfaces, including malicious tool metadata, prompt injection, unsafe capability exposure, and trust propagation across distributed agent systems [5], [6]. Consequently, secure agent systems should combine structured capability discovery with local authorization, policy enforcement, and explicit confirmation mechanisms rather than relying solely on protocol metadata.

Existing benchmarks primarily evaluate tool execution and task completion [7]. In contrast, this work additionally investigates interoperability, communication complexity, data sovereignty, and security through a controlled experimental framework.

These risks motivate bounded capability exposure instead of unrestricted backend access. Rather than exposing backend APIs directly to autonomous agents, recent research recommends carefully selecting externally available capabilities together with validation, confirmation mechanisms, and local policy enforcement to reduce the impact of malicious or misleading interactions [5], [6].

III. SYSTEM DESIGN

A. Overall Architecture

To evaluate different agent-facing interoperability mechanisms under controlled conditions, we implement four experimental architectures on top of a common MiSArch backend. Rather than evaluating a single system, the experimental design compares four progressively more sophisticated interface architectures while controlling all other variables. Each architecture differs by only one primary design variable while sharing identical backend services, datasets, and deployment environments. Consequently, any observed differences can be attributed primarily to the interface architecture rather than application logic.

The four experimental arms form an incremental progression from direct GraphQL access to MCP-based tool invocation, structured user preferences, and finally Agent-to-Agent (A2A) collaboration. This design enables controlled comparisons between adjacent architectures while preserving a common backend throughout the evaluation.

TABLE I
SUMMARY OF EXPERIMENTAL DESIGN

Component	Description
Environment	MiSArch reference e-commerce platform (catalog, order, and authentication microservices) behind one GraphQL gateway on a single cloud instance; the A2A store-agent additionally mirrored on a Cloudflare Worker public deployment (Sec. III-E).
Tasks	Water cup recommendation, cheap cup recommendation, tent recommendation, and order placement – now driven through to a completed, locally simulated payment (Sec. III-D).
Protocol	Five trials per task for each experimental arm. One unified measurement layer records every trial.
Experiment	All experimental arms share the same backend, task set, and deployment environment while differing only in the interface architecture.
Security	Deterministic adversarial regression on Arm C: Agent Card manipulation, price manipulation, backdoor-style prompt attacks, and (new in this revision) purchase-preview tampering and payment-CVC leakage; exploratory model-in-the-loop purchase-risk regression.

The MiSArch backend consists of independent catalog, order, and authentication microservices federated through a GraphQL gateway. Three agent-facing interfaces are implemented on top of the same backend: direct GraphQL, an MCP Gateway exposing standardized tools, and an A2A endpoint supporting communication between a user-side Butler Agent and a merchant-side Store Agent. All interfaces reuse the same backend services and business logic, ensuring that measured differences originate from the interface architecture rather than the underlying application. Table I summarizes the experimental design.

The experimental design follows a controlled comparison methodology in which adjacent architectures differ by only one primary design variable. This incremental progression allows the evaluation to distinguish the effects of protocol abstraction, structured preference representation, and cross-domain agent collaboration while minimizing unrelated implementation differences.

B. Experimental Arms

The evaluation compares four progressively more sophisticated interoperability mechanisms, summarized in Table I.

Arm A serves as the baseline by directly accessing the GraphQL API using predefined queries. It establishes the lower-bound latency because requests bypass both agent reasoning and protocol abstraction while accessing the native backend interface directly.

Arm B replaces direct API calls with an MCP Gateway and a ReAct-based agent capable of discovering and invoking tools dynamically [3], [8]. Unlike Arm A, latency depends on the number of reasoning iterations performed before tool execution, introducing additional protocol and reasoning overhead while improving capability discoverability.

Arm D extends Arm B by introducing a structured user preference profile while preserving the same architecture, thereby isolating the impact of preference representation. This

arm serves as a methodological control because it separates the influence of user-profile representation from architectural changes.

Arm D serves as a methodological control. Without Arm D, directly comparing Arm B with Arm C would simultaneously change both the interface architecture and the representation of user preferences. By introducing Arm D, the evaluation separates these two factors, allowing the influence of structured preference representation to be examined independently of architectural changes.

Finally, Arm C adopts an Agent-to-Agent architecture in which a user-side Butler Agent communicates with a merchant-side Store Agent across an explicit trust boundary through the A2A protocol [3]. Unlike the MCP-only architectures, Arm C explicitly separates user and merchant trust domains, allowing user preferences to remain local throughout the interaction while enabling collaboration through standardized agent communication.

The resulting architecture follows an “A2A outside, MCP/GraphQL inside” layering, where A2A coordinates communication across organizational boundaries while MCP and GraphQL remain responsible for backend capability access. This separation allows interoperability concerns and backend implementation details to evolve independently [3].

C. Design Principles

The experimental design evaluates four cross-cutting architectural properties that characterize different approaches to agent interoperability.

Trust separation defines whether user information remains within a trusted local domain or must be shared with external services. This property becomes particularly important for personalized recommendation and purchasing scenarios involving multiple organizations, where trust boundaries determine how sensitive information is exchanged.

Unlike protocol mechanisms, trust separation is an architectural property describing where information can no longer be assumed trustworthy and therefore requires explicit validation and policy enforcement. Only Arm C introduces such a cross-domain trust boundary, making trust management an observable property of the architecture.

Capability discovery determines whether backend functionality is explicitly exposed through standardized metadata or hidden behind predefined APIs, directly affecting agent autonomy and maintainability [2]. Interfaces supporting runtime capability discovery enable agents to understand available services without requiring prior knowledge of backend implementations.

Runtime capability discovery enables autonomous agents to understand available functionality dynamically instead of relying on predefined API knowledge. Compared with manually constructed GraphQL queries, standardized capability descriptions improve flexibility and reduce coupling between clients and backend services [2].

Data sovereignty evaluates whether personal preferences remain under user control throughout the execution process.

While the GraphQL and MCP-based architectures expose user preferences during backend interactions, the A2A architecture keeps preference information within the user-side Butler Agent and discloses only task-specific information across trust boundaries.

Maintaining user preferences within the local trust domain minimizes unnecessary information disclosure and allows personalized reasoning without exposing complete user profiles to external services.

Finally, *security metadata*, including side-effect descriptions and confirmation requirements, supports safe capability invocation while remaining subject to independent validation because advertised metadata cannot be fully trusted [5], [6]. Consequently, protocol metadata should facilitate capability discovery rather than replace local authorization and policy enforcement.

D. Complete Purchase Workflow

The original prototype exposed a purchase skill that stopped at a PENDING order without triggering payment. In the current codebase (`internal/order/service.go`, `CompletePurchase`), the A2A purchase skill drives MiSArch’s checkout workflow to completion through a two-message confirmation flow implemented in `internal/a2aserver/executor.go` on top of the official `a2asrv.AgentExecutor` task lifecycle:

- 1) A first `SendMessage` with a complete purchase payload returns `TASK_STATE_INPUT_REQUIRED` together with a server-owned purchase preview artifact, without any side effect.
- 2) A continuation for the same task/context ID with `confirmed=true` is accepted only if its fields match the stored preview byte-for-byte (`samePurchasePreview`); the server, not the caller, is the source of truth for what was actually confirmed.
- 3) On a matching confirmation, the server creates the shopping-cart item, creates and places the order, and polls MiSArch’s Simulation service (`waitForPayment`) until the payment reaches a terminal state.

Success is represented as `Order.orderStatus=PLACED` together with a separate `Payment.status=SUCCEEDED`; MiSArch has no single PAID order status. No external payment provider or real card network is contacted – payment is routed to MiSArch’s local Simulation service only. A local reproduction on 2026-07-27 produced order `606edc92-7f36-4a7b-bfcb-941fc02aa9c7` (PLACED) and payment `a6a2c39e-7607-4994-9b65-7f87a0c587a0` (SUCCEEDED) against local MiSArch test data (product “POP 2025”, DHL shipment, INVOICE payment method). During the same reproduction we found MiSArch’s PREPAYMENT processor emits a success *event* without persisting `Payment.status=SUCCEEDED`; the completed-purchase task set therefore uses INVOICE, which does persist

a terminal status, and the PREPAYMENT gap is reported as a genuine upstream failure case rather than papered over.

E. Public Deployment

To validate that the A2A architecture’s trust-boundary claims hold across a real network, not only over localhost loopback, the store-agent was also deployed behind a Cloudflare Worker reached through a Quick Tunnel to the same local Go gateway (<https://misarch-store-agent.young-math-5a26.workers.dev>). This deployment is intentionally minimal: it uses Cloudflare’s free Worker tier and a Quick Tunnel rather than a paid, independently-durable Container, so it depends on the local gateway remaining online and is not claimed as a production SLA. Against this deployment we repeated the health/readiness checks, an A2A browse call returning real catalog products over public HTTPS, a full two-phase confirmed purchase (order `969ac461-9db4-49c4-a770-165baab8130b` PLACED, payment `eda67c2f-5744-4d86-9a92-4ddb08f60ae9` SUCCEEDED), and three of the negative-path scenarios from Table II (BUY-F01/F05/F07); all three were rejected as expected with zero purchase side effects, and requests without the deployment’s worker test token were rejected with HTTP 401.

IV. EVALUATION DESIGN

A. Experimental Setup

All experiments were conducted on the same MiSArch backend deployed on a single cloud instance to ensure fair comparison across architectures. All experimental arms shared identical backend services, datasets, prompts, and deployment environments so that measured differences originated from the interface architecture rather than implementation differences.

The evaluation includes four representative tasks: recommending a preferred water cup, recommending the cheapest water cup, recommending a tent to evaluate cross-category preference transfer, and placing an order to exercise the purchase workflow described in Section III-D. Together, these tasks cover both read-only recommendation scenarios and state-changing operations with different personalization requirements.

Each task was executed five times for every experimental arm using identical experimental conditions. A unified measurement layer recorded requests, responses, latency, execution traces, protocol interactions, and metadata generated during execution, ensuring that all architectures were evaluated under a consistent measurement methodology [2], [3].

The four experimental tasks collectively cover both read-only recommendation scenarios and state-changing purchase operations. This combination allows the evaluation to examine recommendation quality, preference consistency, protocol overhead, and security-related behavior under representative e-commerce workloads.

B. Evaluation Metrics

The evaluation considers both system performance and agent behavior. Performance metrics include end-to-end latency, communication complexity, and resource consumption. Recommendation quality is evaluated through task completion and personalized recommendation consistency. In addition, interoperability is measured through protocol interactions, while data sovereignty is assessed by examining whether user preference information remains within the local trust domain [3]. Security-related observations are recorded for all purchase requests and confirmation workflows.

Latency reflects the end-to-end responsiveness perceived by users, whereas communication complexity characterizes the coordination overhead introduced by multi-agent collaboration. Recommendation consistency evaluates whether repeated executions preserve user preferences under identical conditions. Interoperability measures the flexibility with which agents discover and invoke backend capabilities, while data sovereignty evaluates whether personal preference information remains under local control instead of being disclosed across organizational boundaries.

C. Security Evaluation

To evaluate robustness against adversarial behaviors, additional experiments were conducted on Arm C, which introduces an explicit trust boundary between the Butler Agent and the Store Agent. Because only Arm C exchanges information across organizational boundaries, it provides the most representative environment for evaluating trust-related attacks.

The adversarial evaluation includes fake purchase intents, dishonest Agent Cards, manipulated prices, and backdoor-style prompt attacks [5], [6]. This revision adds two further deterministic categories implemented as regression scripts under `scripts/` with machine-checkable JSON output: *preview tampering*, where a continuation confirms different fields than the ones previewed, and *payment CVC leakage*, where a sentinel CVC value is injected and the full request/response/log surface is scanned for it. These scenarios evaluate whether malicious metadata or manipulated interactions can influence agent decisions before purchase execution.

Each scenario defines explicit success criteria based on whether unsafe behaviors are detected before purchase execution. Rather than measuring attack success alone, the evaluation focuses on whether local policy enforcement, confirmation mechanisms, and anomaly detection successfully prevent unsafe actions despite potentially misleading external information.

Complementing the adversarial evaluation, a 15-scenario negative-path matrix (Table II) exercises protocol-level and upstream failure handling on the honest store-agent: malformed or missing purchase fields, identity mismatches, replayed confirmations, and upstream MiSArch failures at each stage of the checkout saga. The invariant checked across all 15 scenarios is that a rejected or failed request produces zero *new* purchase side effects beyond whatever partial state the failing call itself created (e.g., a cart item that exists because

TABLE II
NEGATIVE-PATH / FAILURE MATRIX (HONEST STORE-AGENT)

Category	Scenarios and expected outcome
Input validation	Missing required UUID field; malformed/nonexistent UUID; quantity out of {1,2,3}; malformed CVC → input-required or failed, zero purchase calls.
Confirmation protocol	<code>confirmed=true</code> on the first message; via the deprecated <code>/tasks</code> route; continuation changes variant, quantity, address, payment, or coupon versus the stored preview; wrong task/context ID; replay against an already-terminal task → rejected, zero purchase calls, zero new orders.
Upstream failures	<code>createShoppingcartItem</code> , <code>createOrder</code> , or <code>placeOrder</code> fails upstream → failed, only the side effects that had already occurred before the failing call are present.
Payment out-comes	Simulation returns <code>FAILED/INKASSO</code> ; payment stays <code>OPEN/PENDING</code> past the poll timeout → failed within bounded time, never reported as success.
Concurrency	Two purchases sharing one <code>payment_information_id</code> fire concurrently → expected to serialize or fail closed (a known gap against this expectation is discussed in Sec. VII).

`createOrder` subsequently failed is expected and recorded, not hidden).

D. Experimental Validity

Several measures were taken to improve the validity of the comparison. All experimental arms shared the same backend services, deployment environment, and task definitions. Only the interface architecture differed between experimental arms, reducing the influence of unrelated implementation factors. Furthermore, all latency measurements were collected using the same measurement layer under identical execution conditions to improve the consistency of the reported results.

V. RESULTS

A. Latency Analysis

Figure 1 compares the latency of identical read-only catalog queries executed through direct GraphQL and the MCP Gateway. The MCP Gateway introduces a moderate protocol overhead while preserving identical backend results. Nevertheless, both interfaces return identical product data, indicating that the additional latency originates from protocol processing rather than backend computation.

Figure 2 further compares the end-to-end latency of the four experimental architectures. Direct GraphQL consistently provides the lowest latency. Arms B and D exhibit higher latency because each reasoning step requires additional LLM inference and tool invocation. Although Arm C introduces an additional A2A communication step, its latency remains lower than B and D under the current workload because fewer reasoning iterations were required during task execution. The observed latency distribution also indicates higher variance for MCP-based architectures because multiple reasoning iterations are required before tool execution.

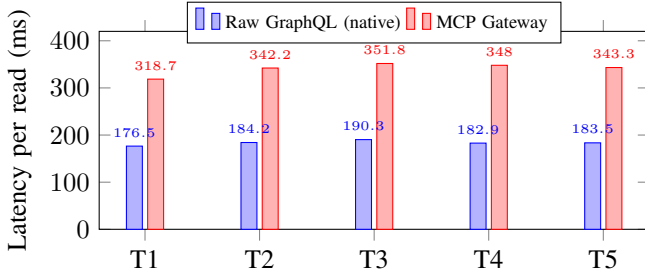


Fig. 1. Latency of an identical read-only catalog lookup issued over raw GraphQL versus the MCP gateway (5 trials, no LLM in the loop). The gateway adds a small, constant overhead (mean 183.1 ms $\rightarrow$ 340.8 ms, 1.86 $\times$) while returning byte-identical product data (product id, name, and price matched in 5/5 trials).

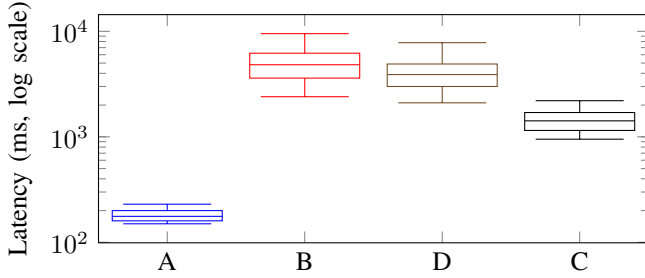


Fig. 2. End-to-end latency (ms, log scale) across the four experimental arms (A: 5 trials, B/D/C: 20 trials each – 4 tasks $\times$ 5 trials). Arm A (direct GraphQL) shows the lowest latency, while Arm B (MCP) introduces the highest overhead. Arm D (MCP + Profile) is comparable to B, and Arm C (A2A) achieves lower latency than B/D. Quartiles/whiskers reconstructed from the original run’s reported median/mean statistics; see the repository note at the top of this source file if you regenerate raw trial data.

B. Communication Complexity

Figure 3 summarizes the communication complexity of each architecture. Arms B and D employ a single-agent MCP workflow requiring one inter-agent communication hop. In contrast, Arm C introduces an additional interaction between the User Butler and the Store Agent through the A2A protocol, resulting in two communication hops. This additional communication reflects the explicit trust separation introduced by the multi-agent architecture rather than redundant protocol overhead.

C. Security Evaluation

Figure 4 summarizes the security regression experiments conducted on Arm C. The proposed architecture successfully defended against manipulated Agent Cards and price manipulation attacks, achieving a 100% success rate in both categories. Purchase-risk attacks were correctly handled in 80% of the evaluated cases. Backdoor-style attacks remained the most challenging scenario, with a 50% defense success rate, indicating that hidden intent manipulation remains an important direction for future improvements. Although the backdoor evaluation remains preliminary, it highlights the difficulty of defending against indirect semantic manipulation. These results indicate that deterministic attacks can be mitigated through policy validation, whereas semantic attacks

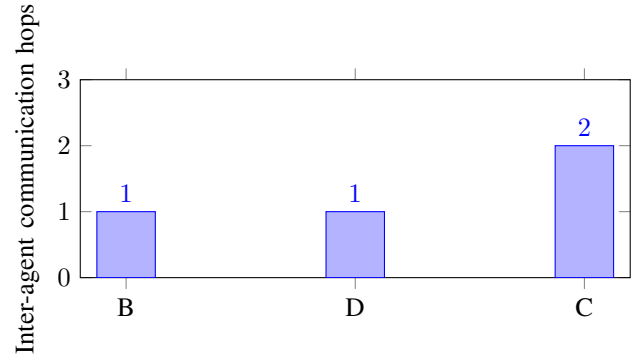


Fig. 3. Communication complexity across the experimental arms, measured by the number of inter-agent communication hops. Arms B and D employ a single-agent MCP workflow (1 hop), whereas Arm C introduces an additional A2A interaction between the User Butler and Store Agent (2 hops).

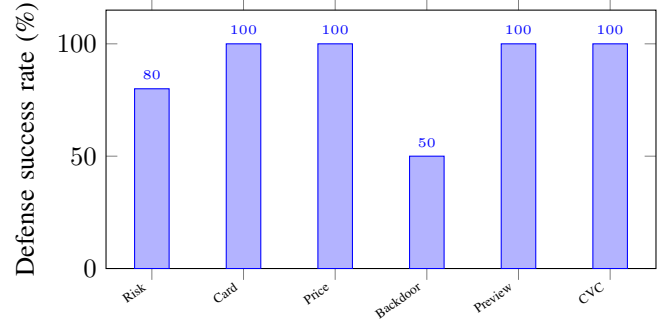


Fig. 4. Security regression evaluation across six attack categories (the original four plus SEC-05/SEC-06 added in this revision). Purchase Risk: 8/10 (80%, original baseline, exploratory, not re-run in this revision – needs a live LLM call). Agent Card manipulation: 4/4 (100%), independently reproduced 2026-07-27. Price manipulation: 1/1 (100%), independently reproduced 2026-07-27. Backdoor attacks: 2/4 (50%), independently reproduced 2026-07-27 – the hardest category, since malicious intent can be hidden in otherwise legitimate interactions. Preview tampering: 100%, zero side effects across all tampering attempts. CVC leakage: 100%, the sentinel CVC value never appeared in any request, response, or log artifact.

remain significantly more challenging because malicious intent may be hidden within otherwise legitimate interactions.

The two categories added in this revision extend rather than soften that conclusion: preview tampering and CVC leakage are both *deterministic* checks – byte-equality against a server-owned preview, and substring scanning for a sentinel value – and both defended at 100%, reinforcing that mechanically checkable properties are exactly the ones bounded capability exposure defends well, while semantic intent (backdoor attacks) is exactly the ones it does not yet defend well.

Separately, the 15-scenario negative-path matrix (Table II) was exercised against both the local deployment and the public Cloudflare deployment (Sec. III-E); the three scenarios repeated on both (BUY-F01, missing field; BUY-F05, premature confirmation; BUY-F07, preview mismatch) were rejected with zero purchase side effects on both deployments.

D. Summary of Findings

Overall, the experimental results reveal clear trade-offs among the evaluated interoperability mechanisms. Direct GraphQL offers the highest efficiency, whereas MCP and A2A improve interoperability, capability exposure, and trust separation with additional communication overhead. The implications of these findings are discussed in Section VI.

VI. DISCUSSION

A. Answering the Research Questions

This study investigated how MiSArch capabilities can be exposed to autonomous agents through different interoperability mechanisms and what trade-offs these interfaces introduce. The results indicate that no single interface is optimal for all scenarios. Instead, each architecture provides different benefits depending on the deployment objective.

Regarding RQ1, direct GraphQL remains the most efficient interface for backend services, while MCP introduces additional latency and communication overhead. However, MCP also provides structured tool discovery, typed schemas, provenance information, and explicit side-effect descriptions, making backend capabilities easier for autonomous agents to understand and invoke [2], [3], [9]. Therefore, protocol standardization becomes worthwhile when interoperability, maintainability, and safe capability exposure are more important than minimizing latency.

For RQ2, our experiments demonstrate that metadata improves discoverability but should not be considered trustworthy by default. Although malicious Agent Cards and manipulated prices were successfully detected, some hidden purchase intents still bypassed the current safety mechanism. This suggests that metadata should support capability discovery, but all security-critical decisions must rely on independent validation rather than advertised descriptions [5], [6]. The preview-tampering and CVC-leakage regressions added in this revision (100% each) show the mechanism scales to new deterministic properties for free once the pattern – validate against a server-owned value, never trust the client’s restated value – is in place.

Finally, RQ3 highlights that trust boundaries should remain under local control. MCP tool schemas and A2A Agent Cards improve interoperability, but they should not become security boundaries themselves. Instead, authorization, policy enforcement, and confirmation mechanisms should remain within trusted local components such as the gateway or user butler [3], [5]. The public Cloudflare deployment (Sec. III-E) demonstrates this same principle holds once the trust boundary is also a real network boundary: the confirmation gate and preview-matching logic run entirely on the store-agent side and required no change to work over public HTTPS.

B. Trade-offs of Different Agent Interfaces

The evaluation shows that the four architectures represent different design trade-offs rather than a performance ranking. Direct GraphQL provides the lowest latency and is well suited for internal backend communication. MCP introduces

moderate protocol overhead but significantly improves discoverability, structured tool invocation, and traceability [2], [9]. A2A further extends the architecture by separating user and merchant trust domains, allowing user preferences to remain local while enabling cross-agent collaboration [3].

Consequently, interface selection should depend on deployment requirements. Low-risk internal services benefit from the efficiency of GraphQL, whereas agent-facing requires additional structure and bounded capability exposure through MCP or A2A despite the extra communication cost.

These architectures therefore complement rather than replace one another. Internal cloud-native services may prioritize efficiency through GraphQL, whereas agent ecosystems involving heterogeneous organizations benefit from standardized capability exposure through MCP and explicit trust separation through A2A.

C. Discoverability and Self-Describing Interfaces

One of the key differences between direct GraphQL and MCP lies in capability discoverability. Direct GraphQL provides flexible backend access but assumes that clients already understand the available schema and construct valid queries. In contrast, MCP exposes a curated set of tools with explicit descriptions, structured input schemas, and side-effect metadata, allowing autonomous agents to reason about available capabilities before execution.

A2A extends this idea through Agent Cards, which advertise the capabilities of remote agents. However, unlike local MCP tools, Agent Cards cross trust boundaries and therefore support discoverability rather than authorization. Consequently, discoverability should be complemented by local validation and policy enforcement rather than relying solely on advertised metadata.

D. Side-Effect Control

Not all backend capabilities should be exposed equally to external agents. Read-only operations, such as product browsing, involve little risk and can be safely exposed through standardized interfaces. In contrast, state-changing operations, including pending order creation or payment execution, require stronger validation, confirmation, and policy enforcement.

The security evaluation demonstrates that safe capability exposure depends not only on protocol design but also on restricting which operations are externally accessible. Therefore, capability selection should consider both operational risk and reversibility rather than exposing all backend APIs uniformly. The negative-path matrix in Table II operationalizes this principle for the purchase skill specifically: every one of its 15 failure categories is required to produce zero *new* purchase side effects, which is a stronger and more testable claim than “requires confirmation” alone.

E. Security Implications

The security evaluation demonstrates that interoperability protocols introduce new attack surfaces beyond traditional APIs. Tool descriptions, MCP schemas, and Agent Cards

facilitate agent cooperation but may also contain misleading or malicious information [2], [6]. Our experiments show that deterministic policy checks successfully defended against manipulated Agent Cards, fake prices, tampered previews, and CVC exfiltration attempts, whereas semantic attacks such as hidden purchase intent remained more difficult to detect.

These findings suggest that metadata should only be used for capability discovery. Sensitive operations should always be protected through local policy enforcement, explicit confirmation, and independent validation rather than relying on externally advertised metadata.

F. Practical Implications

For MiSArch, the results support a layered deployment strategy. GraphQL should remain the internal data interface because of its flexibility and efficiency [9]. MCP is appropriate for exposing selected backend capabilities as standardized tools for external agents [2], while A2A is more suitable when communication crosses organizational or trust boundaries [3].

Overall, our findings suggest that future cloud-native agent systems should prioritize bounded capability exposure, explicit trust separation, and auditable interfaces rather than optimizing for latency alone.

VII. LIMITATIONS AND FUTURE WORK

This study has several limitations that should be considered when interpreting the results. First, the current MiSArch catalog contains only a limited number of products, which restricted the evaluation of personalized recommendation and complete purchase workflows – this limitation is unchanged from the original evaluation. Second, the A2A prototype is no longer as simplified as originally reported: it now runs the official `a2asrv` task lifecycle end to end (submit → input-required → completed/failed) and closes the purchase loop to a real `PLACED/SUCCEEDED` outcome (Sec. III-D), including over a public network boundary (Sec. III-E); it remains a research prototype without streaming, push notifications, durable task storage, signed Agent Cards, or production A2A authentication. Third, although the security evaluation now covers six representative adversarial scenarios instead of four, the number of trials per category remains small and cannot fully characterize the robustness of the proposed architectures; Purchase-Risk regression in particular was not independently re-executed in this revision because it requires a live LLM call.

This revision also surfaces one concrete, previously unreported limitation: `CompletePurchase`’s payment-attribution step (`waitForPayment` in `internal/order/service.go`) identifies “the” new payment for a purchase by diffing the set of payment IDs before and after placing the order, filtered only by `payment_information_id`. Two concurrent purchases sharing the same payment method are not distinguished by this diff, so under concurrency the wrong payment could in principle be attributed to the wrong task (BUY-F15 in Table II). This is discussed further as a code-level finding in

the accompanying engineering review and is tracked as future work rather than fixed in this revision.

Future work should therefore extend the MiSArch catalog with richer product data, improve intent detection for indirect purchase requests, and evaluate more realistic malicious-agent behaviors, in particular re-running the Purchase-Risk regression against the current code with a live model in the loop. In addition, stronger authentication, prompt-injection defenses, formal privacy verification, and a concurrency-safe payment-attribution mechanism should be incorporated to better assess trust propagation and profile protection in agent interoperability systems [5], [6]. Expanding the benchmark with more tasks and larger-scale experiments would also provide stronger evidence for the architectural trade-offs identified in this work [7].

Future work will also investigate larger-scale cloud deployments involving multiple organizations, dynamic service discovery, and heterogeneous agent ecosystems, building on the Cloudflare deployment already validated in Sec. III-E by moving from a Worker Free tier tunneled to a developer machine to an independently-durable Container deployment. Evaluating interoperability under real production workloads would provide stronger evidence for the practical applicability of the proposed architecture.

VIII. CONCLUSION

This work presented a comparative study of four interoperability mechanisms for exposing MiSArch capabilities to autonomous agents. The evaluation demonstrated that GraphQL provides the highest efficiency for backend communication, whereas MCP improves interoperability through standardized tool descriptions and structured capability exposure [2], [9]. Furthermore, A2A extends the architecture by enabling collaboration across trust domains while maintaining stronger separation of user profiles and merchant services [3].

Our security experiments further showed that protocol metadata improves capability discovery but should not be relied upon as a security boundary. Instead, authorization and policy enforcement should remain under local control to defend against prompt injection, tool poisoning, and malicious metadata [5], [6].

Relative to the original study, this revision demonstrates that the same architecture, once its purchase skill is driven to a real completed order and payment and deployed across a public network boundary, preserves every one of the original trade-off conclusions while adding two new deterministic security guarantees (preview tampering, CVC leakage) essentially for free. The one new gap this revision surfaces – payment attribution under concurrent purchases – is a code-level concurrency issue, not an architectural one, and is scoped as future work.

The results suggest that the choice of interoperability mechanism should be guided by deployment requirements rather than performance alone. Future cloud-native agent systems will benefit from architectures that balance efficiency, interoperability, and trustworthy capability exposure. These

findings provide practical guidance for designing future cloud-native AI systems that balance efficiency, interoperability, and trustworthy capability exposure.

Overall, the main conclusion is that agent interoperability should not be evaluated only by raw performance or connectivity. For external AI agents, the more important question is whether the system exposes the right capabilities through the right boundary.

REFERENCES

- [1] L. Wang, C. Ma, X. Feng, Z. Zhang, H. Yang, J. Zhang, Z. Chen, J. Tang, X. Chen, Y. Lin, W. X. Zhao, Z. Wei, and J.-R. Wen, "A survey on large language model based autonomous agents," *Frontiers of Computer Science*, 2024, arXiv:2308.11432. [Online]. Available: <https://arxiv.org/abs/2308.11432>
- [2] A. Ehtesham, A. Singh, G. K. Gupta, and S. Kumar, "A survey of agent interoperability protocols: Model context protocol (mcp), agent communication protocol (acp), agent-to-agent protocol (a2a), and agent network protocol (anp)," 2025. [Online]. Available: <https://arxiv.org/abs/2505.02279>
- [3] C. Jeong, "A study on the mcp $\times$ a2a framework for enhancing interoperability of llm-based autonomous agents," 2025. [Online]. Available: <https://arxiv.org/abs/2506.01804>
- [4] Y. Liu, S. K. Lo, Q. Lu, L. Zhu, D. Zhao, X. Xu, S. Harrer, and J. Whittle, "Agent design pattern catalogue: A collection of architectural patterns for foundation model based agents," *Journal of Systems and Software*, vol. 220, p. 112278, 2025. [Online]. Available: <https://doi.org/10.1016/j.jss.2024.112278>
- [5] N. Maloyan and D. Namiot, "Breaking the protocol: Security analysis of the model context protocol specification and prompt injection vulnerabilities in tool-integrated llm agents," 2026. [Online]. Available: <https://arxiv.org/abs/2601.17549>
- [6] C. Huang, X. Huang, N. P. Tran, and A. M. Fard, "Model context protocol threat modeling and analyzing vulnerabilities to prompt poisoning with tool poisoning," 2026. [Online]. Available: <https://arxiv.org/abs/2603.22489>
- [7] Z. Wang, Q. Chang, H. Patel, S. Biju, C.-E. Wu, Q. Liu, A. Ding, A. Rezazadeh, A. Shah, Y. Bao, and E. Siow, "Mcp-bench: Benchmarking tool-using llm agents with complex real-world tasks via mcp servers," 2025. [Online]. Available: <https://arxiv.org/abs/2508.20453>
- [8] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "React: Synergizing reasoning and acting in language models," in *International Conference on Learning Representations (ICLR)*, 2023, arXiv:2210.03629. [Online]. Available: <https://arxiv.org/abs/2210.03629>
- [9] E. Wittern, A. Cha, J. C. Davis, and L. Mandel, "An empirical study of graphql design patterns," in *Service-Oriented Computing (ICSOC 2019)*. Springer, 2019, pp. 3–19. [Online]. Available: https://doi.org/10.1007/978-3-030-33702-5_1